طرح معماری چند-مشتری، چند-کمپنی و چند-شعبه

وضعیت: **تصویب‌شده، آمادهٔ تطبیق** — نسخهٔ ۳ (ادغام‌شده)
تاریخ تصمیم: ۱۴۰۵/۰۷/۱۹ (2026-10-11)
برنچ در زمان نوشتن: `refactor-items-and-company-wise`

این سند ادغام دو منبع است:

- **سند معماری NextBook نسخهٔ ۱٫۰** — ساختار سازمانی، الگوهای هفت ERP، سناریوهای تجارتی، سطوح گزارش و معیارهای پذیرش.
- **پلان تطبیق نسخهٔ ۲** — وضعیت واقعی کد فعلی، باگ‌های پیش‌نیاز، اندازهٔ مهاجرت و فازبندی اجرایی.

سند معماری می‌گوید **چه بسازیم**؛ این سند می‌گوید **چطور از جایی که امروز هستیم به آنجا برسیم**. نسخهٔ ۱ (مدل تک‌سطحی) به‌کلی رد شده است.

---

۱. خلاصهٔ تصمیم

مدل در یک جمله: یک دیتابیس اختصاصی برای هر مشتری SaaS؛ چند کمپنی مستقل مالی در همان دیتابیس؛ چند شعبه زیر هر کمپنی؛ و گدام، صندوق، مرکز هزینه و مرکز سود به‌عنوان موجودیت‌های مرتبط اما جداگانه.

سه قاعدهٔ بنیادی:

۱. **دو سطح سازمانی:** کمپنی بالای شعبه. کمپنی یعنی یک بیزنس با کتاب‌های مالی مستقل؛ شعبه یعنی یک محل عملیاتی از همان بیزنس. ۲. **مرز اشتراک:** بین شعبه‌های یک کمپنی همه چیز مشترک است — کتلاگ، مشتریان، چارت اکونت، کتاب‌ها — و انتقال جنس یک transfer ساده است. بین کمپنی‌ها **هیچ چیز** مشترک نیست و هر انتقال سند و اثر حسابداری متقابل می‌خواهد. ۳. **SaaS با دیتابیس به‌ازای هر مشتری:** همه چیز روی زیرساخت ما. یک دیتابیس مرکزی فقط اطلاعات سرویس را نگه می‌دارد؛ هیچ سند تجارتی در آن ذخیره نمی‌شود.

---

۲. منشأ الگوها

سیستم	چه چیزی گرفته شد
Odoo	هستهٔ مدل سازمانی: کمپنی‌های مستقل، شعبات، انتخاب کمپنی فعال، قواعد اشتراک دیتا
TallyPrime	سادگی و سرعت ثبت سند، کار با کیبورد، گدام سلسله‌مراتبی، تخصیص مصارف بر اساس فیصدی
Oracle NetSuite	گروه تجارتی، ارز پایهٔ متفاوت هر کمپنی، قواعد معاملات بین‌کمپنی، گزارش تلفیقی در فاز پیشرفته
SAP S/4HANA	تفکیک کمپنی، گدام، موقعیت ذخیره، مرکز سود و مرکز هزینه — گدام و مرکز هزینه با «شعبه» یکی نیستند
ERPNext	ابعاد مالی (شعبه، پروژه، مرکز هزینه) به‌جای تکثیر حساب‌های درآمد و مصرف برای هر واحد
Business Central	اسناد متناظر بین‌کمپنی، نگاشت حساب‌ها، تأیید دوطرفه
Acumatica	ادارهٔ عملیاتی شعبات، صندوق‌ها و مسئولیت‌های مالی هر شعبه

۳. ساختار سازمانی

```
دیتابیس مرکزی (Control Plane)  -- فقط اطلاعات سرویس
tenants, plans, subscriptions, licenses,
coupons, invoices, payments, audit_events

دیتابیس هر مشتری (Data Plane)
[گروه تجارتی]  -- اختیاری، فاز ۲
+-- کمپنی ۰۱ : احمد الکترونیک
|   +-- شعبه: کابل   +-- شعبه: هرات   +-- شعبه: مزار
+-- کمپنی ۰۲ : احمد ساختمانی
    +-- شعبه: کابل   +-- شعبه: قندهار

در هر شعبه: صندوق، گدام، کارمندان، سری اسناد و تنظیمات عملیاتی.
مرکز هزینه و مرکز سود: ابعاد مالی، مستقل از درخت شعبات.
```

موجودیت‌های اصلی

موجودیت	توضیح
`companies`	هویت حقوقی، ارز پایه، دورهٔ مالی، نوع بیزنس، تنظیمات مالی، وضعیت
`branches`	دارای `company_id`، و `parent_id` برای زیرشعبه (این کالم از قبل در کد هست)
`warehouses`, `stock_locations`	مالکیت کمپنی، ارتباط عملیاتی با شعبه
`cash_registers`, `bank_accounts`	مالکیت کمپنی، شعبهٔ مجاز مشخص
`cost_centers`, `profit_centers`, `accounting_dimensions`	طبقه‌بندی مالی، مستقل از درخت شعبات
`user_company_access`, `user_branch_access`	صلاحیت کاربر در کمپنی و شعبه — دو سطح جدا
`intercompany_documents`, `interbranch_transfers`	گردش‌کار اسناد داخلی
`business_groups`	اختیاری، فاز ۲

قاعدهٔ مرز کمپنی

> اگر کتاب‌های مالی، کتلاگ و مشتریان مشترک‌اند → **یک کمپنی با چند شعبه**. اگر مشترک نیستند → **کمپنی‌های جدا**.

در ویزارد یک سؤال می‌شود: «بیزنس تازه است، یا محل دیگری از بیزنس موجود؟»

جواز

هر محل در افغانستان جواز جداگانه لازم دارد، حتی داخل یک بیزنس. پس `license_number` و تاریخ‌های صدور و انقضا روی **شعبه** می‌نشینند، نه کمپنی. در اسناد چاپی، جواز **شعبهٔ صادرکننده** نمایش داده می‌شود.

قواعد یکپارچگی

- هر شعبه دقیقاً متعلق به یک کمپنی است. انتقالش به کمپنی دیگر یک تغییر ساده `company_id` نیست و روند انتقال

تاریخی/قانونی می‌خواهد.

- هر تراکنش مالی حتماً `company_id` دارد، و در عملیات شعبه‌محور `branch_id`.
- ارتباط شعبهٔ کمپنی A با سند کمپنی B باید با composite FK **در سطح دیتابیس** ناممکن شود، نه فقط با کنترل اپلیکیشن.
- هر ثبت نهایی: Actor، زمان، نمبر یکتا در scope مشخص، و سابقهٔ حسابرسی غیرقابل‌ویرایش.
- اگر دامنه نامعلوم بود، درخواست **رد** می‌شود — هرگز به «همهٔ کمپنی‌ها» تبدیل نمی‌شود. انتخاب «همهٔ کمپنی‌ها» فقط در **گزارش‌گیری** مجاز است؛ ثبت، ویرایش و چاپ سند همیشه در یک کمپنی مشخص.

---

۴. مرزهای دیتا

دیتا	سطح
نوع بیزنس، تنظیمات، تقویم، لوکال، ارز پایه، سال مالی	کمپنی
چارت اکونت و انواع اکونت	کمپنی
کتلاگ اجناس و variantها	کمپنی — بین شعبه‌ها مشترک
مشتریان و فروشندگان، سقف اعتبار، مانده	کمپنی — یک مشتری، یک بیلانس
کتگوری، برند، واحد، مقدار	کمپنی
کانفیگ HR (شفت، رخصتی، اجزای معاش، مالیهٔ معاش)	کمپنی
جواز و تاریخ تجدید	شعبه
صندوق و حساب بانکی	کمپنی (مالکیت) + شعبهٔ مجاز
سری نمبر سند	کمپنی + نوع سند + شعبه + دوره
موجودی، lot، piece	گدام
مرکز هزینه، مرکز سود، پروژه	بُعد مالی، مستقل از درخت شعبه
هر تراکنش	`company_id` اجباری + `branch_id` در عملیات شعبه‌محور

ابعاد مالی به‌جای تکثیر حساب

از ERPNext: برای تفکیک سود و مصرف هر شعبه، **حساب جدید نسازید**. یک حساب «فروشات» با بُعد شعبه کافی است. تکثیر حساب‌ها به‌ازای هر واحد، چارت اکونت را در یک سال غیرقابل استفاده می‌کند.

---

۵. لایهٔ SaaS

Control Plane — دیتابیس مرکزی

```
tenants          نام، subdomain، شناسهٔ امن اتصال دیتابیس، منطقهٔ استقرار،
                 وضعیت (active / grace / read_only / archived)، plan_code
code
plans            نام، قیمت، دورهٔ صورتحساب، features، limits
tenant_id، plan_code
subscriptions    وضعیت، پایان دورهٔ جاری
tenant_id، valid_from، valid_until، revoked_at
licenses
code
coupons          نوع (درصد/مبلغ/روز)، مقدار، سقف استفاده، اعتبار
```

```
coupon_redemptions   کوپون، مشتری، اشتراک، زمان
invoices / payments  صورتحساب و پرداخت
audit_events         چه کسی روی کدام مشتری چه کرد
```

هیچ سند فروش، مشتری تجارتی، موجودی یا حسابداری در این دیتابیس ذخیره نمی‌شود. کلید اتصال دیتابیس در Secret Manager نگه داشته شود و هرگز در رابط کاربر نمایش داده نشود.

Data Plane — دیتابیس اختصاصی مشتری

پس از خرید سرویس، دیتابیس اختصاصی ساخته و migrationهای نسخه‌بندی‌شده اجرا می‌شوند. بکاپ و Restore فقط روی دیتابیس همان مشتری انجام می‌شود.

کالم `tenant_id` در دیتابیس مشتری ساخته نمی‌شود. دیتابیس خودش مرز مشتری است؛ کالمی که در تمام ردیف‌ها یک مقدار دارد صفر معلومات حمل می‌کند و فقط حس امنیت کاذب می‌سازد. لایسنس هم در دیتابیس مشتری ذخیره نمی‌شود — entitlement از مرکز خوانده و کش می‌شود.

مسیر امن درخواست

۱. کاربر وارد می‌شود و هویتش احراز می‌گردد. ۲. سرور Tenant مجاز را **از هویت و عضویت معتبر** تعیین می‌کند — نه از پارامتر مرورگر. ۳. سرور اتصال دیتابیس همان Tenant را انتخاب می‌کند. ۴. کمپنی و شعبهٔ انتخاب‌شده با صلاحیت کاربر تطبیق داده می‌شوند. ۵. در هر query، ثبت و گزارش، دامنهٔ کمپنی/شعبه اعمال می‌شود.

عملیات SaaS

بکاپ و بازیابی مستقل هر مشتری، migration قابل‌ردیابی و قابل‌بازیابی، پایش سلامت، محدودیت منابع، و غیرفعال‌سازی امن اشتراک.

سقف واقعی تعداد connection است، نه تعداد دیتابیس. PgBouncer از همان اول در پلان زیرساخت باشد.

بلاک کردن باید نرم باشد

```
active -> grace (هشدار، همه چیز کار می‌کند)
read_only <- (دیدن و اکسپورت بله، ثبت سند نه)
archived <- (فقط اکسپورت)
```

هیچ‌وقت حذف، هیچ‌وقت قفل کامل. دکانی که وسط ماه از کتاب‌های خودش بیرون بماند دیگر شما را به کسی معرفی نمی‌کند، و `read_only` از نظر فشار پرداخت تقریباً به همان اندازه مؤثر است.

دو منبع جدا برای «آیا این ماژول روشن است؟»

```
[enabled = business_profile.modules[x]   <- دواخانه به تولیدی ضرورت ندارد
[AND license.features[x]                 <- این مشتری HR را نخریده
```

دو سؤال متفاوت با دو جواب متفاوت. اگر از اول جدا نشوند، بعداً گره می‌خورند.

---

۶. امنیت و ایزولاسیون

> **Database-per-Tenant فقط جداسازی مشتریان را می‌دهد.** جداسازی کمپنی‌ها داخل دیتابیس یک مشتری همچنان به فیلترهای اجباری، مجوزهای سروری، کلیدهای ارتباطی درست و تست‌های امنیتی ضرورت دارد.

مرز امنیتی از `WHERE` به **resolver اتصال** منتقل می‌شود. یک scope فراموش‌شده یک جدول را لو می‌دهد؛ یک باگ در انتخاب connection **کل دیتای یک مشتری** را. پس بیشترین تست باید آنجا باشد.

سه خطری که همین حالا در کد وجود دارد

هر سه امروز تصادفاً کار می‌کنند چون driverها دیتابیسی‌اند (`CACHE_STORE=database`، `QUEUE_CONNECTION=database`)، و روزی که برای مقیاس SaaS به Redis بروید **هر سه هم‌زمان و بی‌صدا می‌شکنند**:

۱. **ضمیمه‌ها بدون مرز مشتری.** در `AttachmentService` فایل با `('file->store($directory, 'public$` روی دیسک **public** می‌نشیند — وب‌سرور مستقیم سرو می‌کند، بدون چک دسترسی و بدون مشتری در مسیر. در SaaS یعنی اسناد همهٔ مشتریان در یک پوشهٔ عمومی با URL قابل حدس. **اصلاح:** مسیر `.../{tenants/{tenant`، دیسک **private**، دانلود مجازشده.

۲. **کلیدهای کش بدون پیشوند مشتری.** کلیدهایی مثل `main_branch`، `gl_accounts`، `items`، `currencies`، `branches` رشته‌های سراسری بدون دامنه‌اند. روی Redis مشترک، لیست اسعار مشتری A به مشتری B سرو می‌شود. **اصلاح:** پیشوند مشتری در `CacheKey` و عبور دادن تمام کلیدهای خام از آن.

۳. **jobها زمینهٔ مشتری را حمل نمی‌کنند.** هیچ‌کدام از ۱۲ job موجود `tenant_id` ندارند. با job، Redis queueهای همهٔ مشتریان در یک صف می‌ریزند. **اصلاح:** هر job مشتری را حمل کند و اتصال را دوباره برقرار کند.

لایهٔ دوم: RLS پستگرس

global scopeهای Eloquent تنها خط دفاع نباشند. یک کوئری SQL خام یا یک scope فراموش‌شده می‌تواند نشت بدهد. Row Level Security روی جدول‌های پرارزش فعال شود: نقش اپلیکیشن مالک جدول نباشد، و زمینه در تراکنش تنظیم و در اتصال‌های مشترک پاک شود.

کنترل سمت سرور

هر API باید Branch، Company، Tenant و نوع عملیات را اعتبارسنجی کند. **مخفی کردن یک گزینه در UI جایگزین authorization سروری نیست.** تغییر دستی `company_id` در درخواست نباید دسترسی ایجاد کند. مجوز در زمان اجرای job دوباره بررسی شود. دسترسی به بکاپ، خروجی گزارش و فایل‌های پیوست فقط در محدودهٔ مجاز.

---

۷. حسابداری

هر کمپنی دفتر کل، AR/AP، صندوق، بانک و بستن دوره‌اش مستقل است. **یک سند حسابداری نمی‌تواند خطوط دو کمپنی داشته باشد.** توازن بدهکار و بستانکار در ارز عملیاتی همان کمپنی، هنگام قطعی‌سازی بررسی می‌شود.

انتقال بین شعبه‌های یک کمپنی

گدام کابل ۲۰ لپ‌تاپ به گدام هرات می‌فرستد. یک «انتقال موجودی» با وضعیت ارسال/دریافت ساخته می‌شود. چون هر دو به یک کمپنی تعلق دارند، **این فروش نیست**: خروج، موجودی در راه، و رسید گدام ثبت می‌شود. ارزش طبق روش قیمت‌گذاری همان کمپنی

حفظ می‌شود و هزینهٔ حمل جداگانه تخصیص می‌یابد.

این دقیقاً همان قابلیتی است که کل این طرح برای آن ساخته شده.

معامله بین دو کمپنی یک مشتری

احمد الکترونیک به احمد ساختمانی تجهیزات می‌فروشد: سند فروش در کمپنی اول، سند خرید در کمپنی دوم، با `intercompany_transaction_id` مشترک، تأیید دوطرفه و نگاشت حساب مناسب. **نه یک انتقال معمولی بین گدام‌ها.** شکست طرف دوم نباید به‌عنوان موفقیت کامل نمایش داده شود.

تقسیم مصارف مشترک

کرایهٔ دفتر مرکزی ۳۰٬۰۰۰ افغانی: ۵۰٪ کابل، ۳۰٪ هرات، ۲۰٪ مزار. **تغییر فیصدی باید نسخه‌بندی یا تاریخ مؤثر داشته باشد**، تا گزارش‌های گذشته بی‌دلیل تغییر نکنند.

ارز

ارز پایه برای هر کمپنی مشخص است؛ ارز معامله می‌تواند AFN، USD، PKR یا ارز مجاز دیگر باشد. هر سند: مبلغ ارزی، نرخ ثبت‌شده، تاریخ نرخ و معادل ارز پایه را نگه می‌دارد. **نرخ سند گذشته با تغییر نرخ امروز عوض نمی‌شود.** برای حواله: منبع نرخ، تاریخ، کارمزد، مرجع حواله و وضعیت تسویه ثبت شود — حوالهٔ در جریان، پول دریافت‌شده نیست.

واحدها

کارتن، عدد، بوجی، کیلوگرام و سیر با ضریب تبدیل دقیق مدیریت شوند.

---

۸. صلاحیت‌ها و سطوح گزارش

نقش‌ها

نقش	دامنه
Tenant Owner	مدیریت اشتراک، کمپنی‌ها و ادمین‌ها؛ دسترسی مالی مطابق مجوز صریح
Group Finance	گزارش و عملیات مجاز روی چند کمپنی مشخص
Company Admin	مدیریت کاربران و اطلاعات کمپنی خودش
Branch Manager	عملیات و گزارش شعبه‌های مجاز
Cashier / Warehouse Operator	دسترسی محدود به صندوق یا گدام مشخص
Platform Admin	دسترسی پشتیبانی بین‌مشتری — یک flag روی کاربر، با ثبت کامل

دسترسی در **دو سطح** داده می‌شود: `user_company_access` و `user_branch_access`. یک کاربر می‌تواند در یک کمپنی باشد ولی فقط به دو شعبه‌اش دسترسی داشته باشد.

سطوح گزارش

۱. **شعبه** — فروش، مصارف، صندوق و موجودی همان شعبه ۲. **کمپنی** — گزارش مالی کامل و مقایسهٔ شعبات ۳. **گروه** — گزارش مدیریتی چندکمپنی؛ تلفیق مالی با حذف معاملات داخلی در فاز ۳ ۴. **Tenant** — داشبورد جمع‌بندی با رعایت scope کاربر، بدون اختلاط دیتای حقوقی مستقل

۹. تجربهٔ کاربری

پیچیدگی پنهان می‌ماند تا وقتی لازم شود

مشتری SaaS ثبت‌نام می‌کند. سیستم در دیتابیس اختصاصی‌اش **یک کمپنی، یک شعبهٔ اصلی، یک گدام و یک صندوق پیش‌فرض** می‌سازد. رابط کاربری پیچیدگی گروه و چندکمپنی را **پنهان می‌کند** — ولی دیتامدل از روز نخست آمادهٔ رشد است.

یک دکان کوچک هرگز نباید کلمهٔ «کمپنی» و «شعبه» را ببیند. سویچر کمپنی فقط وقتی ظاهر می‌شود که کمپنی دوم ساخته شود؛ انتخاب شعبه فقط وقتی که شعبهٔ دوم اضافه شود.

تعویض کمپنی بدون اشتباه

نام کمپنی و شعبهٔ فعال همیشه در باالی صفحه و **روی خود فرم** دیده شود. هنگام تعویض با فرم ذخیره‌نشده، هشدار داده شود. زمینهٔ کمپنی **در میانهٔ یک درخواست تغییر نکند** — یک متغیر عمومی نشست نباید کمپنی تب دیگر را عوض کند.

---

۱۰. وضعیت فعلی کد و فاصله تا این طرح

این بخش در سند معماری نیست و مهم‌ترین بخش برای تطبیق است. سند معماری طوری نوشته شده که انگار از صفر می‌سازیم؛ در واقع یک ERP کامل موجود است.

آنچه امروز هست

- جدول `companies` وجود دارد، ولی **tenant نیست** — یک ردیف تنظیمات و پروفایل است که به کاربر وصل شده (`preferences`، `business_type`، تقویم، لوکال).
- جدول `branches` وجود دارد و **عملاً نقش کمپنی را بازی می‌کند**: `BranchProvisioningService` به هر شعبه چارت اکونت کامل، ست اسعار با ارز پایه، financial period، کتلاگ، کتگوری، برند، واحد، گدام و مشتری نقده می‌دهد — حدود ۲۰۰ ردیف به‌ازای هر شعبه.
- `branches.parent_id` از قبل وجود دارد → زیرشعبه مجانی است.
- `branches` هیچ `company_id` ندارد → **حفرهٔ امنیتی:** `SwitchBranchController` فقط `exists:branches,id` را چک می‌کند، پس یک super-admin می‌تواند به شعبهٔ هر کمپنی دیگر سویچ کند.
- `company_id` فقط روی `users` و `invoice_formats` است؛ ۹۶ جدول دیگر فقط `branch_id` دارند. ۵۳ unique index شامل `branch_id` است.
- `business_profiles` موتور نوع بیزنس از قبل کار می‌کند — **تمایز اصلی محصول**، و باید حفظ شود.

فاصلهٔ اصلی

چون امروز هر شعبه چارت اکونت و کتلاگ **خودش** را دارد، برای اینکه شعبه‌های یک کمپنی بتوانند دیتا را شریک شوند و جنس بین‌شان منتقل شود، این نسخه‌ها باید در یکی **ادغام** شوند. این یعنی بازنویسی `transaction_lines.account_id` روی دیتای پست‌شده — تنها جایی در کل طرح که می‌تواند بی‌صدا تریل بیلانس را جابه‌جا کند.

اندازهٔ این خطر را هیچ‌کس هنوز نمی‌داند. اگر اکثر مشتریان یک شعبه داشته باشند، ادغام تقریباً بی‌خطر است. اولین کاری که باید

نوشته شود کامند `company:audit` است که همین را جواب بدهد.

---

۱۱. باگ‌ها و بدهی‌های پیش‌نیاز

مورد	چرا پیش‌نیاز است
`HasSequentialNumber` از `MAX+1` استفاده می‌کند	دو فروش هم‌زمان همان نمبر را می‌گیرند. سند معماری هم صریحاً `MAX+1` را منع کرده
`sale_items` اسنپ‌شات جنس ندارد — فقط `item_id`	وقتی کتلاگ بین شعبه‌ها مشترک شد، تغییر نام یک جنس **ظاهر فاکتورهای گذشته را عوض می‌کند**
`Company` بدون `SoftDeletes` ولی `restore` route دارد	۵۰۰ می‌دهد
`Company::search()` بدون trait `HasSearch`	صفحهٔ لیست کمپنی‌ها می‌شکند
`tenant_id` مرده در `Account` و `Currency`	کالم در هیچ migration ساخته نشده
`branches.sub_domain` بلااستفاده	در SaaS، subdomain مال مشتری است نه شعبه
`BranchSpecific` وقتی برنچ bind نشده از scope بیرون می‌زند	در job و console، ردیف‌های همه را می‌بیند

---

۱۲. فازها

فازهای سند معماری (MVP / پیشرفته / سازمانی) با واقعیت کد ترکیب شده‌اند:

فاز	محتوا	خطر
۰ — سنجش و پاک‌کاری	کامند `company:audit` (چند شعبه زیر هر کمپنی در دیتای واقعی؟) · رفع هر هفت مورد بخش ۱۱ · رفع سه خطر امنیتی بخش ۶	کم
۱ — هستهٔ SaaS	دیتابیس مرکزی Provisioning · Tenant Resolver + Secret Manager + `tenants` و migration روی همهٔ دیتابیس‌ها · قلاب entitlement · کش، فایل و job مشتری-آگاه · تست عدم نشت بین مشتریان	متوسط
۲ — کمپنی به‌عنوان سطح	صفات بیزنس روی `branches.company_id` · `companies` + جواز · `company_id` روی ~۹۶ جدول با composite FK · `SetActiveCompany` · محدود کردن سویچ شعبه — **حفرهٔ امنیتی اینجا بسته می‌شود** · RLS	متوسط
۳ — ادغام master data	چارت اکونت، کتلاگ، مشتریان، اسعار و واحدهای شعبه‌های یک کمپنی در یکی · اسنپ‌شات روی خط سند · تجزیهٔ provisioning به کمپنی/شعبه	**بالا**
۴ — انتقال بین شعبه‌ها	`interbranch_transfers` با وضعیت ارسال/دریافت + اکونت موجودی در راه + هزینهٔ حمل	کم
۵ — دسترسی و سویچر	`user_company_access` و `user_branch_access` · سویچر کمپنی و شعبه · ویزارد ساخت با پیش‌نمایش نوع بیزنس · پنهان‌سازی پیچیدگی برای مشتری تک‌کمپنی	کم
۶ — تنظیمات و ماژول‌ها	Settings resolver (`config` → business profile → کمپنی → یوزر) · `modules` AND features شده با لایسنس · Spatie teams روی `company_id`	متوسط
۷ — ابعاد مالی	`accounting_dimensions`، `profit_centers`، `cost_centers` · تقسیم مصارف با فیصدی و تاریخ مؤثر · گزارش مقایسه‌ای شعبات	کم
۸ — معاملات بین‌کمپنی	`intercompany_documents` با شناسهٔ مشترک، تأیید دوطرفه، نگاشت حساب، اکونت‌های due-to/due-from	متوسط
۹ — گروه و تلفیق	`business_groups` · گزارش مدیریتی گروه · در فاز پیشرفته تلفیق با حذف معاملات درون‌گروهی	متوسط
۱۰ — پنل مدیریت مشتریان	پکیج، اشتراک، کوپون، فاکتور، بلاک و تمدید	کم

فاز ۳ تنها فازی است که دیتای مالی را جابه‌جا می‌کند. فاز ۲ هم ۵۳ unique index را بازنویسی می‌کند. هر دو باید اول روی بازگردانی دیتابیس بزرگ‌ترین مشتری تمرین شوند.

شرط انتشار فاز ۱: تست عدم نشت دیتا بین مشتریان و کمپنی‌ها.

---

۱۳. معیار صحت مهاجرت و آزمون پذیرش

قبل و بعد از فاز ۳ باید دقیقاً تطابق کنند

- تعداد اسناد و خطوط
- جمع بدهکار و بستانکار
- ماندهٔ هر مشتری و فروشنده
- مقدار و ارزش موجودی
- ماندهٔ صندوق، بانک و دارایی‌ها

و این سه باید **صفر** باشند: رکورد بدون کمپنی، مرجع خارج از دامنه، سند بدون خط معتبر.

آزمون پذیرش

سناریو	نتیجهٔ لازم
کاربر مشتری A با دستکاری شناسه	هیچ راهی برای خواندن یا نوشتن دیتای مشتری B
تعویض `company_id` در API	رد دسترسی؛ هیچ عنوان، مبلغ یا ضمیمه‌ای افشا نشود
کاربر شعبهٔ هرات	اسناد کابل را نبیند و تغییر ندهد، مگر مجوز صریح
انتخاب مشتری یا گدام کمپنی دیگر	رد در سرور **و** دیتابیس، حتی اگر UI دور زده شود
دو تب با دو کمپنی	هر تب فقط در کمپنی خودش ثبت و چاپ کند
فروش در کمپنی A	بر ماندهٔ کمپنی B اثر نگذارد، مگر از مسیر ثبت‌شدهٔ intercompany
Retry پرداخت یا صدور فاکتور	سند دوم ایجاد نشود؛ نمبر تکراری صادر نشود
انتقال بین دو شعبهٔ یک کمپنی	عاید نسازد؛ موجودی در راه درست ثبت شود
گزارش مجموعی شعبات	با جمع معاملات و تعدیلات کمپنی سازگار باشد
حذف، غیرفعال‌سازی یا Restore یک مشتری	بر دیتابیس سایر مشتریان اثر نگذارد
Migration نسخهٔ جدید روی چند مشتری	قابل‌ردیابی، قابل‌پایش و در صورت خطا قابل‌بازیابی
ارز و دورهٔ بسته	نرخ تاریخی حفظ شود؛ قطعی‌سازی در دورهٔ بسته رد شود
دری، پشتو و چاپ	RTL، تاریخ، اعداد، نام کمپنی و شعبه در PDF درست

---

۱۴. تصمیمات ثبت‌شده

تصمیم	چرا
دو سطح (کمپنی > شعبه)، نه یک سطح	مدل تک‌سطحی نمی‌تواند انتقال و دیتای مشترک بین شعبه‌ها بدهد

تصمیم	چرا
`tenant_id` روی جدول‌های مشتری اضافه نشود	دیتابیس خودش مرز است؛ کالم ثابت صفر معلومات دارد و حس امنیت کاذب می‌سازد
لایسنس در دیتابیس مشتری ذخیره نشود	SaaS است؛ مرکز همیشه در دسترس است
چارت اکونت در سطح کمپنی، نه مشترک بین کمپنی‌ها	هر کمپنی کتاب‌های جداگانه دارد
کتلاگ در سطح کمپنی، مشترک بین شعبه‌ها	نیازمندی صریح
جواز روی شعبه	هر محل در افغانستان جواز جدا لازم دارد
تفکیک سود شعبه با **بُعد مالی**، نه حساب جدید	تکثیر حساب، چارت را در یک سال غیرقابل‌استفاده می‌کند
گدام ≠ شعبه ≠ مرکز هزینه	سه موجودیت با سه هدف؛ یکی کردنشان بعداً قابل جدا کردن نیست
`business_groups` در فاز ۹	هر دو سند موافق‌اند که فاز ۱ نیست
هویت مرجع مشترک مشتری در سطح Tenant — فعلاً نه	با «دیتا بین کمپنی‌ها مشترک نباشد» در تضاد است؛ اگر ثبت تکراری مشتری به شکایت تبدیل شد، بازبینی شود
`business_type` enum + config بماند	«اضافه کردن صنف نباید معنایش migration باشد» — و این تمایز اصلی محصول است
بدون فیلد مالیهٔ فروش	در بازار افغانستان ثبت مالیاتی نیست؛ یک تنظیم با پیش‌فرض صفر کافی است
کار آفلاین شعبه یک پروژهٔ جداست	نتیجهٔ خودکار Database-per-Tenant نیست؛ به حل تعارض و sync مستقل ضرورت دارد

---

۱۵. چالش‌های آینده

۱. **معاملات بین‌کمپنی ناگزیر است.** مالک از صندوق یک بیزنس، قرض بیزنس دیگر را می‌دهد. فاز ۸ شروع کار است، نه پایانش. ۲. **رپورت ترکیبی خواسته می‌شود حتی بدون مالیات.** چون چارت‌ها جدا هستند، به نقشه‌برداری بین چارت‌ها ضرورت دارد. **هِج ارزان:** مطمئن شوید چارت پیش‌فرض هر کمپنی عیناً همان slugها را نگه می‌دارد، تا جمع‌زدن فردا یک `group by slug` باشد. ۳. **تجربهٔ کاربری با تعداد کمپنی خراب می‌شود. هِج ارزان:** scope را از روز اول `(whereIn('company_id', $ids` بنویسید با آرایهٔ یک‌عضوی. ۴. `business_type` **به‌عنوان config تا جایی مقیاس می‌گیرد.** تولیدی به BOM و هوتل به اتاق ضرورت دارد — این‌ها flag نیستند، موجودیت‌اند. ۵. **دیتابیس-به-ازای-مشتری یعنی هر تغییر schema ضرب در تعداد مشتری.** نسخه‌ها واگرا می‌شوند و پشتیبانی تبدیل می‌شود به «این مشتری روی کدام نسخه است؟» ۶. **ردیف کمپنی به god-object تنظیمات تبدیل می‌شود.**

فایده‌ای که این مدل ممکن می‌کند

دیتابیس مرکزی می‌تواند چیزهای مشترک را به همهٔ مشتریان پوش کند. ملموس‌ترین نمونه برای این بازار: **نرخ روزانهٔ افغانی/دالر/کلدار** یک بار مرکزی ثبت شود و به همهٔ مشتریان برسد، با امکان override برای هر مشتری.

---

۱۶. سؤالات باز

۱. در دیتای واقعی مشتریان، چند شعبه زیر هر کمپنی وجود دارد؟ این اندازهٔ ادغام فاز ۳ و کل خطر پروژه را تعیین می‌کند. ۲. نمبرگذاری سند: سری به‌ازای کمپنی با پیشوند شعبه (`-INV-KBL`)، یا سری مستقل به‌ازای هر شعبه؟ ۳. آیا `read_only` برای مشتری بلاک‌شده کافی است؟ ۴. صندوق: مالکیت کمپنی با شعبهٔ مجاز، یا کاملاً متعلق به شعبه؟ (سند معماری اولی را می‌گوید؛ کد فعلی دومی را فرض کرده.)

۱۷. منابع

- Odoo Documentation — Multi-company
- TallyPrime — Cost Centre و گدام سلسله‌مراتبی
- Oracle NetSuite — OneWorld، گروه و تلفیق
- SAP S/4HANA — Company Code، Plant، Profit/Cost Center
- ERPNext — Accounting Dimensions
- Microsoft Dynamics 365 Business Central — Intercompany Workflows
- Acumatica — Branch Accounting
- PostgreSQL — Row Level Security
- IFRS IAS 21 (ارز) و IFRS 10 (تلفیق)